

ГУАП

КАФЕДРА № 42

ОТЧЕТ  
ЗАЩИЩЕН С ОЦЕНКОЙ  
ПРЕПОДАВАТЕЛЬ

старший преподаватель  
\_\_\_\_\_  
должность, уч. степень, звание

\_\_\_\_\_  
подпись, дата

Д. О. Шевяков  
\_\_\_\_\_  
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 7

СОЗДАНИЕ И НАСТРОЙКА СИСТЕМЫ СБОРА ДАННЫХ НА БАЗЕ  
ПЛАТФОРМЫ ИНТЕРНЕТА ВЕЩЕЙ

по курсу:

ИНТЕРНЕТ ВЕЩЕЙ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4326

\_\_\_\_\_  
подпись, дата

Г. С. Томчук  
\_\_\_\_\_  
инициалы, фамилия

Санкт-Петербург 2026

## **1 Цель работы**

Цель работы: приобретение навыков создания системы долгосрочного хранения данных.

## **2 Задание**

Создать систему логирования для долгосрочного хранения данных. Итоговый класс должен содержать, как минимум два различных метода для сохранения данных в разные коллекции.

Вариант задания: «Умная теплица».

## **3 Ход выполнения**

В ходе выполнения лабораторной работы была подготовлена программная часть системы сбора и долгосрочного хранения данных для системы умной теплицы. Для обеспечения долговременного хранения однотипных записей была выбрана документо-ориентированная база данных MongoDB, а в файл зависимостей проекта был добавлен модуль pymongo, необходимый для подключения к MongoDB из Python.

Была реализована архитектура доступа к данным через отдельный класс базы данных Database, отвечающий за хранение параметров подключения, создание клиента MongoDB и получение статуса соединения. В методе проверки состояния соединения было реализовано выполнение команды ping, получение списка коллекций в базе данных, а также подсчет количества документов в коллекциях и общего количества записей. Фрагмент кода реализации класса Database и метода получения статуса соединения изображен на рисунке 1.

```

8 class Database:
9     def __init__(
10         self,
11         *,
12         mongo_uri: str = "mongodb://localhost:27017/",
13         db_name: str = "greenhouse_db",
14     ):
15         self.mongo_uri = mongo_uri
16         self.db_name = db_name
17         self._client = pymongo.MongoClient(self.mongo_uri)
18         self._db = self._client[self.db_name]
19
20     @property
21     def db(self):
22         return self._db
23
24     def get_status(self) → Dict[str, Any]:
25         try:
26             self._client.admin.command("ping")
27             collections = list[str](self._db.list_collection_names())
28             counts: Dict[str, int] = {}
29             total = 0
30             for name in collections:
31                 c = int(self._db[name].estimated_document_count())
32                 counts[name] = c
33                 total += c
34             return {
35                 "connection": "ok",
36                 "db_name": self.db_name,
37                 "uri": self.mongo_uri,
38                 "collections": counts,
39                 "records_total": total,
40             }
41         except Exception as exc:
42             return {
43                 "connection": "error",
44                 "db_name": self.db_name,
45                 "uri": self.mongo_uri,
46                 "error": str(exc),
47                 "collections": {},
48                 "records_total": 0,
49             }

```

Рисунок 1 — Обновленный класс Database

Был создан отдельный класс логирования `Logger`, использующий экземпляр `Database` как слой доступа к MongoDB. Было реализовано два метода сохранения: один для записи показаний датчиков в коллекцию `SensorReadings`, второй для записи событий и состояний исполнительных устройств в коллекцию `DeviceEvents`. Для предотвращения избыточного роста базы данных было реализовано исключение дублирующихся записей: перед вставкой новых данных выполняется проверка на изменение значения относительно последнего сохраненного, а при отсутствии изменений вставка пропускается. Фрагмент кода реализации класса `Logger` с двумя методами сохранения и логикой дедупликации изображен на рисунке 2.

```

9  class Logger:
10     def __init__(self, database: Database):
11         self._database = database
12         self._db = database.db
13
14         # кеш последнего записанного значения, чтобы не плодить дубликаты
15         self._last: Dict[Tuple[str, str], Any] = {}
16
17     def _should_insert(self, category: str, key: str, new_value: Any) → bool:
18         cache_key = (category, key)
19         if self._last.get(cache_key) == new_value:
20             return False
21         self._last[cache_key] = new_value
22         return True
23
24     def get_status(self) → Dict[str, Any]:
25         status = self._database.get_status()
26         status["dedupe_cache_size"] = len(self._last)
27         return status
28
29     def insert_sensor_reading(
30         self,
31         *,
32         sensor_id: int,
33         sensor_name: str,
34         sensor_type: str,
35         value: float,
36         unit: str,
37         status: bool,
38         timestamp: Optional[datetime] = None,
39     ):
40         """
41         Записать показание датчика (без дублей по значению).
42         Коллекция: SensorReadings
43         """
44         key = f"{sensor_id}:{sensor_type}"
45         if not self._should_insert("sensor", key, value):
46             return None
47
48         doc = {
49             "timestamp": (timestamp or datetime.now()),
50             "sensor": {
51                 "id": sensor_id,
52                 "name": sensor_name

```

Рисунок 2 — Класс Logger

Далее логирование было интегрировано в серверную часть на Flask. В файле приложения был создан единый экземпляр Database для подключения к MongoDB и экземпляр Logger, использующий этот объект базы данных. В обработчиках запросов опроса датчиков после получения новых значений были добавлены вызовы метода сохранения показаний датчиков, а после выполнения автоматического управления были добавлены вызовы метода сохранения событий исполнительных устройств для фиксации текущего состояния вентилятора и насоса. Фрагмент кода интеграции логирования в обработчики запросов Flask изображен на рисунке 3.

```

18 database = Database(mongo_uri="mongodb://localhost:27017", db_name="greenhouse_logs")
19 db_logger = Logger(database)
20
21 last_temperature = 0.0
22 last_soil = 0.0
23
24 temperature_sensor.turn_on()
25 humidity_sensor.turn_on()
26 soil_sensor.turn_on()
27 fan.turn_on()
28 pump.turn_on()
29
30
31 @app.route("/")
32 def index():
33     return render_template("dashboard.html")
34
35
36 @app.route("/connect/temperature")
37 def connect_temperature():
38     global last_temperature
39     data = temperature_sensor.connect()
40     last_temperature = data["value"]
41     controller.auto_control(float(last_temperature), float(last_soil))
42     db_logger.insert_sensor_reading(
43         sensor_id=int(data["id"]),
44         sensor_name=str(data["name"]),
45         sensor_type=str(data["type"]),
46         value=float(data["value"]),
47         unit=str(data["unit"]),
48         status=bool(data["status"]),
49     )
50     db_logger.insert_device_event(
51         device_id=fan.id,
52         device_name=fan.name,
53         device_type=fan.type,
54         status=bool(fan.status),
55         power=float(fan.power),
56         event="auto_control",
57         details={"reason": "temperature update"},
58     )
59     db_logger.insert_device_event(
60         device_id=pump.id,
61         device_name=pump.name,
62         device_type=pump.type,
63         status=bool(pump.status),
64         power=float(pump.power),
65         event="auto_control",
66         details={"reason": "temperature update"},
67     )
68     return jsonify(data)

```

Рисунок 3 — Вызов функций логирования в БД

Для контроля работоспособности системы хранения и удобства проверки результата в пользовательском интерфейсе был доработан блок отображения состояния базы данных. Было обеспечено отображение реального состояния соединения с MongoDB, имени используемой базы данных, общего количества записей и разбивки по коллекциям, получаемых из эндпоинта мониторинга. Обновленный вид блока состояния БД, содержащий расширенную информацию о состоянии MongoDB и количестве сохраненных документов, изображен на рисунке 4.

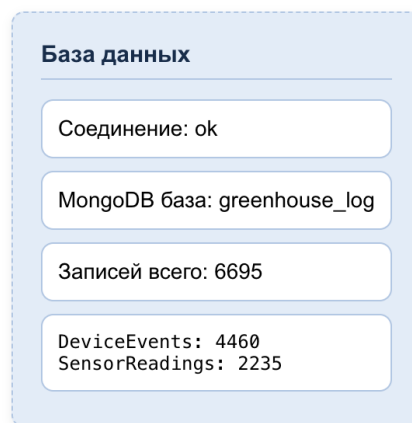


Рисунок 4 — Обновленный блок состояния БД

#### 4 Выводы

В результате выполнения лабораторной работы была реализована система долгосрочного хранения данных IoT-приложения на базе MongoDB. Было обеспечено сохранение показаний датчиков и событий исполнительных устройств в отдельные коллекции, что упрощает последующий анализ и просмотр истории работы системы. Для снижения объема избыточных данных была реализована проверка на повтор значений, благодаря чему в базу заносятся только изменения.

Также была реализована проверка состояния соединения с базой данных и вывод диагностической информации (доступность MongoDB, имя базы, перечень коллекций и количество документов). Это позволило повысить наблюдаемость системы и обеспечить быстрое выявление проблем подключения. В итоге созданное решение демонстрирует практический подход к построению подсистемы хранения и мониторинга данных в IoT-проектах и может быть расширено дополнительными типами событий и метрик без изменения общей архитектуры.

## ПРИЛОЖЕНИЕ

### Исходный код

Исходный код приложения также доступен по ссылке:

<https://github.com/grigoriytomczuk/smart-greenhouse>.

#### Листинг 1 — app.py

```
import logging

from flask import Flask, jsonify, render_template, request

from db import Database
from devices import Actuator, MainControlUnit, Sensor
from logger import Logger

app = Flask(__name__)

temperature_sensor = Sensor(1, "Датчик температуры", "temperature", "°C",
"temperature")
humidity_sensor = Sensor(2, "Датчик влажности воздуха", "humidity", "%", "humidity")
soil_sensor = Sensor(3, "Датчик влажности почвы", "soil_moisture", "%", "soil")
fan = Actuator(101, "Вентилятор", "fan", 120.0)
pump = Actuator(102, "Насос полива", "pump", 80.0)
controller = MainControlUnit(fan_device=fan, pump_device=pump)

database = Database(mongo_uri="mongodb://localhost:27017", db_name="greenhouse_logs")
db_logger = Logger(database)

last_temperature = 0.0
last_soil = 0.0

temperature_sensor.turn_on()
humidity_sensor.turn_on()
soil_sensor.turn_on()
fan.turn_on()
pump.turn_on()

@app.route("/")
def index():
    return render_template("dashboard.html")

@app.route("/connect/temperature")
def connect_temperature():
    global last_temperature
    data = temperature_sensor.connect()
    last_temperature = data["value"]
    controller.auto_control(float(last_temperature), float(last_soil))
    db_logger.insert_sensor_reading(
        sensor_id=int(data["id"]),
        sensor_name=str(data["name"]),
        sensor_type=str(data["type"]),
        value=float(data["value"]),
        unit=str(data["unit"]),
        status=bool(data["status"]),
    )
    db_logger.insert_device_event(
```

```

        device_id=fan.id,
        device_name=fan.name,
        device_type=fan.type,
        status=bool(fan.status),
        power=float(fan.power),
        event="auto_control",
        details={"reason": "temperature update"},
    )
    db_logger.insert_device_event(
        device_id=pump.id,
        device_name=pump.name,
        device_type=pump.type,
        status=bool(pump.status),
        power=float(pump.power),
        event="auto_control",
        details={"reason": "temperature update"},
    )
    return jsonify(data)

@app.route("/connect/soil")
def connect_soil():
    global last_soil
    data = soil_sensor.connect()
    last_soil = data["value"]
    controller.auto_control(float(last_temperature), float(last_soil))
    db_logger.insert_sensor_reading(
        sensor_id=int(data["id"]),
        sensor_name=str(data["name"]),
        sensor_type=str(data["type"]),
        value=float(data["value"]),
        unit=str(data["unit"]),
        status=bool(data["status"]),
    )
    db_logger.insert_device_event(
        device_id=fan.id,
        device_name=fan.name,
        device_type=fan.type,
        status=bool(fan.status),
        power=float(fan.power),
        event="auto_control",
        details={"reason": "soil update"},
    )
    db_logger.insert_device_event(
        device_id=pump.id,
        device_name=pump.name,
        device_type=pump.type,
        status=bool(pump.status),
        power=float(pump.power),
        event="auto_control",
        details={"reason": "soil update"},
    )
    return jsonify(data)

@app.route("/connect/humidity")
def connect_humidity():
    data = humidity_sensor.connect()
    db_logger.insert_sensor_reading(
        sensor_id=int(data["id"]),
        sensor_name=str(data["name"]),
        sensor_type=str(data["type"]),

```



```

        value=float(data["value"]),
        unit=str(data["unit"]),
        status=bool(data["status"]),
    )
    return jsonify(data)

@app.route("/connect/fan")
def connect_fan():
    data = fan.connect()
    db_logger.insert_device_event(
        device_id=int(data["id"]),
        device_name=str(data["name"]),
        device_type=str(data["type"]),
        status=bool(data["status"]),
        power=float(data["power"]),
        event="manual_poll",
    )
    return jsonify(data)

@app.route("/connect/pump")
def connect_pump():
    data = pump.connect()
    db_logger.insert_device_event(
        device_id=int(data["id"]),
        device_name=str(data["name"]),
        device_type=str(data["type"]),
        status=bool(data["status"]),
        power=float(data["power"]),
        event="manual_poll",
    )
    return jsonify(data)

@app.route("/connect/controller")
def connect_controller():
    return jsonify(controller.connect())

@app.route("/connect/database")
def connect_database():
    return jsonify(database.connect())

@app.route("/connect/command", methods=["GET"])
def connect_command():
    """Применить управляющие параметры ко всем вещам."""
    return jsonify(
        {
            "temperature": temperature_sensor.connect(request),
            "humidity": humidity_sensor.connect(request),
            "soil": soil_sensor.connect(request),
            "fan": fan.connect(request),
            "pump": pump.connect(request),
            "controller": controller.connect(request),
            "database": database.connect(request),
        }
    )

if __name__ == "__main__":

```

```
logging.getLogger("werkzeug").setLevel(logging.ERROR)
app.run(debug=True)
```

## Листинг 2 — db.py

```
import re
from typing import Any, Dict, Optional

import pymongo
from werkzeug.wrappers import Request

class Database:
    def __init__(
        self,
        *,
        mongo_uri: str = "mongodb://localhost:27017/",
        db_name: str = "greenhouse_db",
    ):
        self.mongo_uri = mongo_uri
        self.db_name = db_name
        self._client = pymongo.MongoClient(self.mongo_uri)
        self._db = self._client[self.db_name]

    @property
    def db(self):
        return self._db

    def get_status(self) -> Dict[str, Any]:
        try:
            self._client.admin.command("ping")
            collections = list(self._db.list_collection_names())
            counts: Dict[str, int] = {}
            total = 0
            for name in collections:
                c = int(self._db[name].estimated_document_count())
                counts[name] = c
                total += c
            return {
                "connection": "ok",
                "db_name": self.db_name,
                "uri": self.mongo_uri,
                "collections": counts,
                "records_total": total,
            }
        except Exception as exc:
            return {
                "connection": "error",
                "db_name": self.db_name,
                "uri": self.mongo_uri,
                "error": str(exc),
                "collections": {},
                "records_total": 0,
            }

    def _apply_control(self, request: Request) -> None:
        cmd = request.args.get("database_command", "").strip()
        if not cmd:
            return
        if re.match(r"^[a-zA-Z0-9_\-]+$", cmd):
            print(f"[LOG] MongoDB: принята управляющая команда '{cmd}'")
        else:
```

```

        print(
            f"[LOG] Ошибка: команда БД '{cmd}' содержит недопустимые символы.
Разрешены буквы, цифры, '_', '-'"
        )

    def connect(self, request: Optional[Request] = None) -> Dict[str, Any]:
        if request is not None:
            self._apply_control(request)
            status = self.get_status()
            if status.get("connection") == "ok":
                print(
                    f"[LOG] MongoDB OK: db={status.get('db_name')}, всего
записей={status.get('records_total')}}"
                )
            else:
                print(f"[LOG] MongoDB ERROR: {status.get('error')}")
        return status

```

### Листинг 3 — logger.py

```

from __future__ import annotations

from datetime import datetime
from typing import Any, Dict, Optional, Tuple

from db import Database

class Logger:
    def __init__(self, database: Database):
        self._database = database
        self._db = database.db

        # кеш последнего записанного значения, чтобы не плодить дубликаты
        self._last: Dict[Tuple[str, str], Any] = {}

    def _should_insert(self, category: str, key: str, new_value: Any) -> bool:
        cache_key = (category, key)
        if self._last.get(cache_key) == new_value:
            return False
        self._last[cache_key] = new_value
        return True

    def get_status(self) -> Dict[str, Any]:
        status = self._database.get_status()
        status["dedupe_cache_size"] = len(self._last)
        return status

    def insert_sensor_reading(
        self,
        *,
        sensor_id: int,
        sensor_name: str,
        sensor_type: str,
        value: float,
        unit: str,
        status: bool,
        timestamp: Optional[datetime] = None,
    ):
        """
        Записать показание датчика (без дублей по значению).

```

```

Коллекция: SensorReadings
"""
key = f"{sensor_id}:{sensor_type}"
if not self._should_insert("sensor", key, value):
    return None

doc = {
    "timestamp": (timestamp or datetime.now()),
    "sensor": {
        "id": sensor_id,
        "name": sensor_name,
        "type": sensor_type,
        "unit": unit,
        "status": status,
    },
    "value": value,
}

try:
    return self._db["SensorReadings"].insert_one(doc)
except Exception as exc: # pragma: no cover
    print(f"[LOGGER] Ошибка записи SensorReadings: {exc}")
    return None

def insert_device_event(
    self,
    *,
    device_id: int,
    device_name: str,
    device_type: str,
    status: bool,
    power: Optional[float] = None,
    event: str = "state",
    details: Optional[Dict[str, Any]] = None,
    timestamp: Optional[datetime] = None,
):
    """
    Записать событие/состояние исполнительного устройства (без дублей по
    состоянию).
    Коллекция: DeviceEvents
    """
    dedupe_value = {
        "status": status,
        "power": power,
        "event": event,
        "details": details,
    }
    key = f"{device_id}:{device_type}"
    if not self._should_insert("device", key, dedupe_value):
        return None

    doc = {
        "timestamp": (timestamp or datetime.now()),
        "device": {
            "id": device_id,
            "name": device_name,
            "type": device_type,
        },
        "event": event,
        "status": status,
        "power": power,
        "details": details or {},
    }

```

```

}

try:
    return self._db["DeviceEvents"].insert_one(doc)
except Exception as exc: # pragma: no cover
    print(f"[LOGGER] Ошибка записи DeviceEvents: {exc}")
    return None

```

#### Листинг 4 — index.js

```

...
function getDatabaseData() {
    $.ajax({
        type: "GET",
        url: "/connect/database",
        dataType: "json",
        contentType: "application/json",
        data: {},
        success: function (response) {
            $("#db_connection").val("Соединение: " + response.connection);
            if (response.db_name)
                $("#db_name").val("MongoDB база: " + response.db_name);
            else $("#db_name").val("MongoDB база: -");

            if (typeof response.records_total !== "undefined")
                $("#db_records").val("Записей всего: " + response.records_total);
            else $("#db_records").val("Записей всего: -");

            if (response.collections) {
                const entries = Object.entries(response.collections);
                entries.sort((a, b) => (b[1] || 0) - (a[1] || 0));
                const lines = entries.map(([k, v]) => `${k}: ${v}`);
                $("#db_collections").val(lines.join("\n") || "Коллекций пока нет");
            } else $("#db_collections").val("Коллекций: -");
        },
    });
}
...

```